

Exception Handling: Checked vs Unchecked + Custom Exceptions

1. What is an Exception?

An exception is an event that disrupts the normal execution of a program.

Example:

```
int result = 10 / 0;
```

This causes:

```
ArithmeticException
```

Instead of allowing the application to fail unexpectedly, we handle the condition appropriately.

2. Exception Hierarchy

Know this basic hierarchy:

```
Throwable
├── Error
│   ├── OutOfMemoryError
│   └── StackOverflowError
└── Exception
    ├── RuntimeException
    │   ├── NullPointerException
    │   ├── IllegalArgumentException
    │   └── ...
    └── Other checked exceptions
        ├── IOException
        ├── SQLException
        └── ...
```

Important

Error and **Exception** are different.

Generally:

- **Error** → serious JVM/system-level problem; application usually should not try to recover from it.
 - **Exception** → condition an application may be able to handle.
-

3. Checked Exceptions

A **checked exception** is checked by the compiler at compile time.

Examples:

```
IOException  
SQLException  
ClassNotFoundException
```

Example:

```
public void readFile() throws IOException {  
    ...  
}
```

You must either:

Handle it

```
try {  
    readFile();  
} catch (IOException e) {  
    ...  
}
```

Or declare it

```
public void process() throws IOException {  
    readFile();  
}
```

4. Why are they called “checked”?

Because the compiler checks whether you have dealt with the exception.

Example:

```
public void test() {  
    FileInputStream input =  
        new FileInputStream("file.txt");  
}
```

This will not compile without handling/declaring the checked exception.

The compiler effectively says:

“You must deal with this possibility.”

5. Unchecked Exceptions

Unchecked exceptions are subclasses of `RuntimeException`.

Examples:

```
NullPointerException
IllegalArgumentException
IllegalStateException
ArithmeticException
IndexOutOfBoundsException
```

You don't have to explicitly catch or declare them.

```
public void calculate() {
    int x = 10 / 0;
}
```

Compiles, but fails at runtime.

6. Checked vs Unchecked

Feature	Checked	Unchecked
Parent	Exception excluding RuntimeException	RuntimeException
Compiler checks	Yes	No
Must catch/declare	Yes	No
Typical examples	IOException, SQLException	NPE, IllegalArgumentException
Often represents	Recoverable/external conditions	Programming errors or invalid state/input

7. Which Should You Use?

This is where interviews become more interesting.

Don't say:

“Always use checked exceptions.”

or:

“Always use runtime exceptions.”

The decision depends on the situation and application architecture.

Checked exception can make sense when:

The caller can reasonably be expected to handle the failure.

Example:

```
File doesn't exist
External resource unavailable
```

Unchecked exception is often preferable for:

- invalid arguments
- invalid application state

- business exceptions
- programming errors

Modern Spring applications commonly use **unchecked custom exceptions** for business-layer failures.

8. Custom Exceptions

A custom exception is an exception you create to represent an application-specific failure.

For example, in your BPS payment application:

```
public class PaymentNotFoundException
    extends RuntimeException {

    public PaymentNotFoundException(String message) {
        super(message);
    }
}
```

Then:

```
Payment payment = repository.findById(id)
    .orElseThrow(() ->
        new PaymentNotFoundException(
            "Payment not found: " + id
        ));
```

This is better than throwing a generic:

```
throw new RuntimeException("Not found");
```

because the exception communicates the **actual business meaning**.

9. More Useful Payment Exceptions

For your project, examples might include:

```
PaymentNotFoundException
PaymentAlreadyProcessedException
InvalidPaymentException
PaymentProcessingException
PaymentGatewayException
DuplicateTransactionException
```

For example:

```
public class PaymentAlreadyProcessedException
    extends RuntimeException {

    public PaymentAlreadyProcessedException(
        String message) {
        super(message);
    }
}
```

Then:

```
if (payment.isAlreadyProcessed()) {  
    throw new PaymentAlreadyProcessedException(  
        "Payment already processed"  
    );  
}
```

10. Why Custom Exceptions?

Suppose you do this everywhere:

```
throw new RuntimeException("Something went wrong");
```

Now your controller has no reliable way to distinguish:

```
Payment not found  
Duplicate payment  
Invalid amount  
Gateway failure  
Database failure
```

Custom exceptions give you semantic separation.

```
PaymentNotFoundException  
    ↓  
404  
  
PaymentAlreadyProcessedException  
    ↓  
409  
  
InvalidPaymentException  
    ↓  
400 / 422  
  
PaymentGatewayException  
    ↓  
502 / 503
```

The exact HTTP mapping should match your API contract.

11. Global Exception Handling in Spring Boot

This is **very important for a 3-year Spring Boot interview**.

Instead of writing:

```
try {  
    ...  
} catch (...) {  
    ...
```

```
}
```

in every controller, use:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

}
```

Example:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(PaymentNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
        PaymentNotFoundException ex) {

        ErrorResponse response =
            new ErrorResponse(
                "PAYMENT_NOT_FOUND",
                ex.getMessage()
            );

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(response);
    }
}
```

Now:

```
Controller
↓
Service
↓
throw PaymentNotFoundException
↓
@RestControllerAdvice
↓
HTTP 404
```

12. Why @RestControllerAdvice?

Without centralized handling:

```
Controller 1 → try/catch
Controller 2 → try/catch
Controller 3 → try/catch
```

This creates duplicated and inconsistent code.

With global exception handling:

```
All controllers
↓
Global Exception Handler
↓
Consistent API error response
```

This gives:

- centralized handling
- consistent error responses
- cleaner controllers
- easier maintenance

13. Custom Error Response

A common structure:

```
public class ErrorResponse {  
  
    private String code;  
    private String message;  
    private String traceId;  
  
    // constructors/getters/setters  
}
```

Response:

```
{  
  "code": "PAYMENT_NOT_FOUND",  
  "message": "Payment not found",  
  "traceId": "abc-123"  
}
```

For production systems, avoid exposing stack traces or sensitive internal details to clients.

Log the details internally.

14. throw vs throws

Very common interview question.

throw

Used to actually throw an exception.

```
throw new PaymentNotFoundException(  
    "Payment not found"  
);
```

throws

Used in a method signature to declare that a method may propagate an exception.

```
public void process() throws IOException {  
    ...  
}
```

Remember:

```
throw
→ actually throws

throws
→ declares possibility
```

15. try-catch-finally

```
try {
    // code that may fail
} catch (Exception e) {
    // handle exception
} finally {
    // cleanup
}
```

`finally` generally executes whether an exception occurs or not, subject to abnormal JVM termination scenarios.

Modern Java also provides **try-with-resources** for resources such as files or streams.

16. Try-with-resources

Instead of:

```
FileInputStream input = null;

try {
    input = new FileInputStream("data.txt");
    ...
} finally {
    if (input != null) {
        input.close();
    }
}
```

Use:

```
try (FileInputStream input =
    new FileInputStream("data.txt")) {
    ...
}
```

The resource is automatically closed.

Know this for interviews.

17. Don't Catch Exception Everywhere

Avoid:


```
try {
    processPayment();
} catch (Exception e) {
    return "Payment failed";
}
```

This can hide serious problems.

Instead, catch exceptions when you can actually:

- recover
- transform
- add useful context
- return an appropriate error

Otherwise let the exception propagate to a centralized handler.

18. Exception Chaining

Suppose your database layer throws:

```
SQLException
```

You want to expose a meaningful application-level exception:

```
throw new PaymentProcessingException(
    "Unable to persist payment",
    ex
);
```

Custom exception:

```
public class PaymentProcessingException
    extends RuntimeException {

    public PaymentProcessingException(
        String message,
        Throwable cause) {

        super(message, cause);
    }
}
```

This preserves the original cause.

You can then log the full root cause internally.

19. Don't Leak Sensitive Information

Bad API response:

```
{
  "message": "SQLException: user=root password=..."
}
```

Never expose this kind of information.

Better:

```
{
  "code": "PAYMENT_PROCESSING_ERROR",
  "message": "Unable to process payment",
  "traceId": "TX-123"
}
```

Internally:

```
Application logs
→ Full exception + stack trace
→ Trace ID
→ Transaction ID
```

20. Business vs Technical Exceptions

This is **very important for your BPS project**.

Business exception

The system is working correctly, but a business rule prevents the operation.

Examples:

```
Payment already processed
Insufficient balance
Invalid payment state
Payment not found
```

Technical exception

The system cannot complete the operation because of infrastructure/technical failure.

Examples:

```
Database unavailable
Payment gateway timeout
Network failure
Connection refused
```

Don't mix these indiscriminately.

21. Example Payment Flow

Suppose:

```
POST /payments
↓
Controller
↓
Service
```

```
↓  
Validate payment  
↓  
Process payment  
↓  
Gateway
```

Possible failures:

Invalid amount

```
throw new InvalidPaymentException(...);
```

Already processed

```
throw new PaymentAlreadyProcessedException(...);
```

Gateway timeout

```
throw new PaymentGatewayException(...);
```

DB failure

Database/infrastructure exception may propagate through your technical exception handling strategy.

Then:

```
Exception  
↓  
@RestControllerAdvice  
↓  
ErrorResponse  
↓  
HTTP status
```

22. Critical Payment-System Scenario

Interviewer:

“The payment gateway times out. What exception would you throw?”

Don't immediately say:

```
throw new RuntimeException();
```

Think about the business state.

A timeout does **not necessarily mean the payment failed**.

It might be:

```
BPS  
↓  
Gateway
```

```
↓  
Payment processed successfully  
↓  
Response lost  
↓  
BPS sees timeout
```

Therefore you may have an **unknown/indeterminate transaction state**.

Your system might need:

```
Timeout  
↓  
Mark transaction as PROCESSING / UNKNOWN  
↓  
Don't blindly retry  
↓  
Check/reconcile transaction
```

The exact flow depends on your actual payment architecture.

This is a very strong project-level interview topic.

23. Exception Handling Best Practices

For a production Spring Boot application:

Do

```
Use meaningful custom exceptions  
Centralize REST exception handling  
Return consistent error responses  
Log useful diagnostic context  
Preserve root cause  
Don't expose internal details  
Separate business and technical failures
```

Avoid

```
catch (Exception) everywhere  
empty catch blocks  
generic "Something went wrong"  
stack traces in API responses  
using exceptions for normal control flow
```

24. Interview Questions

Prepare these.

Basic

1. What is an exception?
2. Checked vs unchecked exception?
3. What is `RuntimeException`?

4. What is the difference between `throw` and `throws`?
5. What is `finally`?
6. Can `finally` fail to execute?
7. What is try-with-resources?

Custom exceptions

8. What is a custom exception?
9. Why create custom exceptions?
10. Checked or unchecked custom exception—which would you choose?
11. How do you create a custom exception?
12. How do you preserve the original cause?

Spring Boot

13. What is `@ExceptionHandler`?
14. What is `@ControllerAdvice`?
15. `@ControllerAdvice` vs `@RestControllerAdvice`?
16. How do you implement global exception handling?
17. How do you return different HTTP status codes for different exceptions?
18. How do you handle validation exceptions?

Project-level

19. What exceptions did you create in BPS?
20. Which are business exceptions?
21. Which are technical exceptions?
22. How do you handle downstream API failures?
23. How do you handle timeout?
24. How do you handle DB failure?
25. How do you prevent sensitive information from appearing in logs/API responses?
26. How do you trace an exception to a particular transaction?

25. Interview-ready answer

If they ask:

“Explain checked and unchecked exceptions and how you handle exceptions in your Spring Boot project.”

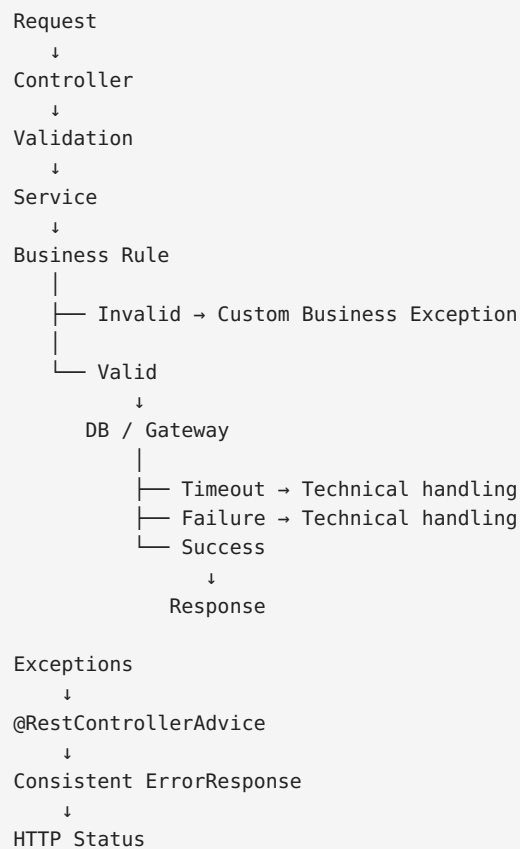
A strong 3-year answer:

“Checked exceptions are checked by the compiler, so they must either be caught or declared, with examples like `IOException`. Unchecked exceptions extend `RuntimeException` and don’t require explicit handling, such as `IllegalArgumentException` or `NullPointerException`.

In a Spring Boot application, I generally use meaningful unchecked custom exceptions for business failures—for example, `PaymentNotFoundException` or `PaymentAlreadyProcessedException`. These exceptions propagate from the service layer and are handled centrally using `@RestControllerAdvice` and `@ExceptionHandler`, which converts them into consistent HTTP responses. For technical failures, such as downstream timeout or database issues, we use appropriate technical exception handling and preserve the original cause for diagnosis. We also avoid exposing stack traces or sensitive internal

details to API clients.”

For your BPS interview, remember this flow



Highest-priority concepts for you

Checked vs unchecked → custom exceptions → throw vs throws → exception propagation → @ExceptionHandler → @RestControllerAdvice → business vs technical exceptions → exception handling for payment gateway timeout/failure.

The last one is particularly important because it lets you connect **Java exception handling to your actual BPS payment project**, which is exactly the kind of cross-question you should expect at 3 years.